Sistema de Gestión de inventario para una ferretería – TPI Programación 1 de la carrera de Programación a Distancia de la UTN

Autor: Reta Reina Victor Ignacio, Comisión A26 C1-13

Descripción General:

Este documento se realizó para explicar exactamente cómo se desarrolló la solución digital de un software de inventario para que una ferretería local pudiera hacer control del stock de sus productos en tiempo real y evitar pérdidas de mercancía.

Contexto del problema:

Una ferretería local necesita digitalizar el control de sus productos para evitar pérdidas de stock. Actualmente manejan sus datos de forma manual y requieren un programa que permita gestionar las herramientas a la venta y sus unidades disponibles en tiempo real.

Este software es necesario para optimizar el tiempo del personal del negocio y reducir al mínimo el error humano asociados al conteo y registro manual de la mercancía.

Enlace al video explicativo y al repositorio de GitHub:

Youtbe: https://www.youtube.com/watch?v=nfF_3H7a6aQ&t=382s

GitHub: https://github.com/VIC-TH0R/UTN---hardware-store-project

Solución Actual e Interacción del Usuario:

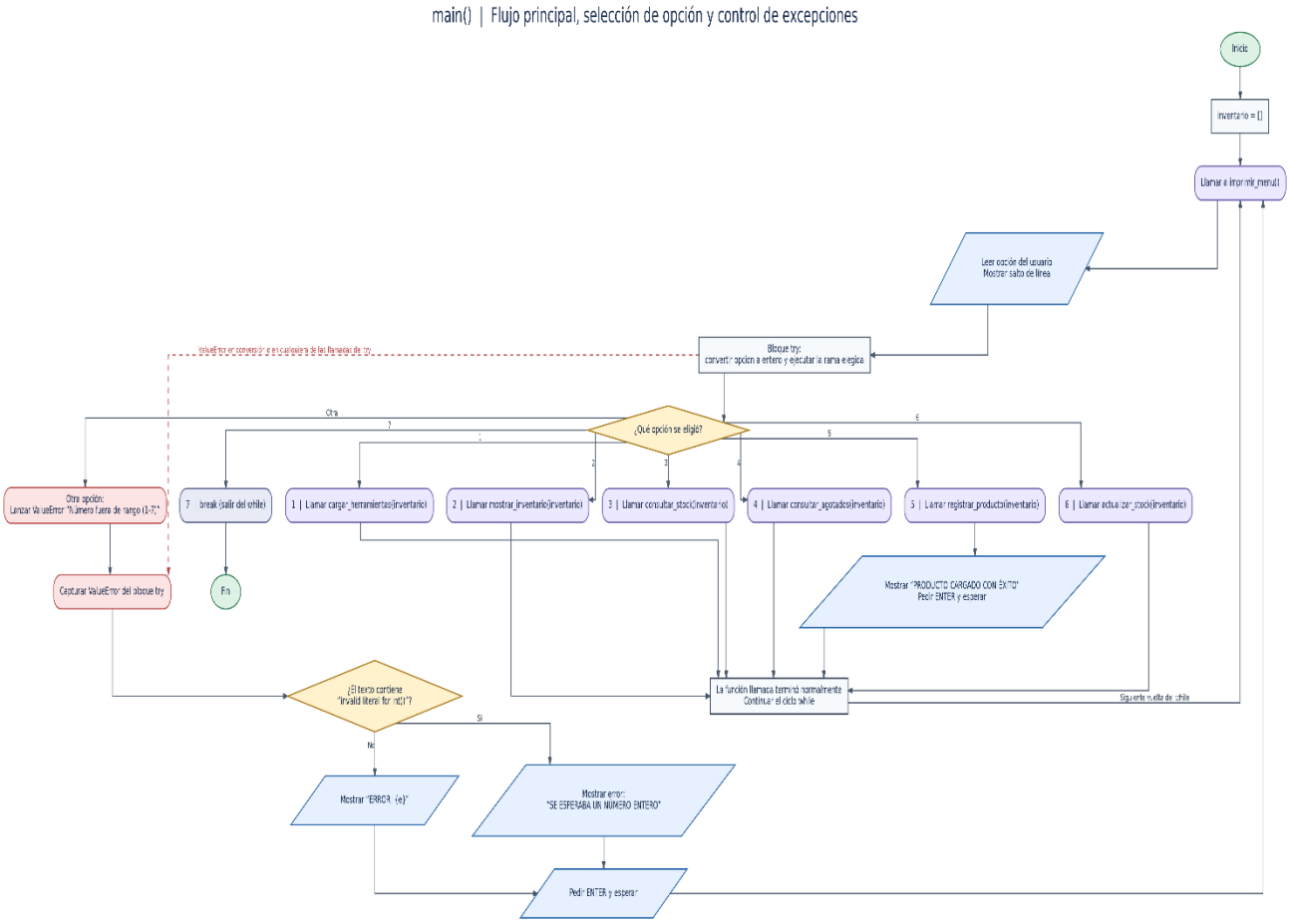
- **Situación Actual:** El negocio opera de forma manual. El control del stock se realiza mediante anotaciones en papel o revisiones visuales directas en los estantes. Esto genera retrasos al atender clientes, pérdidas de material no detectadas a tiempo y errores humanos en los recuentos.
- **Caso de Uso Principal (Flujo del Usuario):**
  Para ilustrar cómo resolverá esto el software, el flujo de interacción del encargado de la ferretería seguirá estos pasos:

1. **Inicio de jornada:** El sistema inicializa la estructura del inventario vacía directamente en la memoria RAM. El usuario puede realizar una carga masiva inicial de herramientas mediante la Opción 1 del menú, quedando el sistema listo para operar en tiempo real

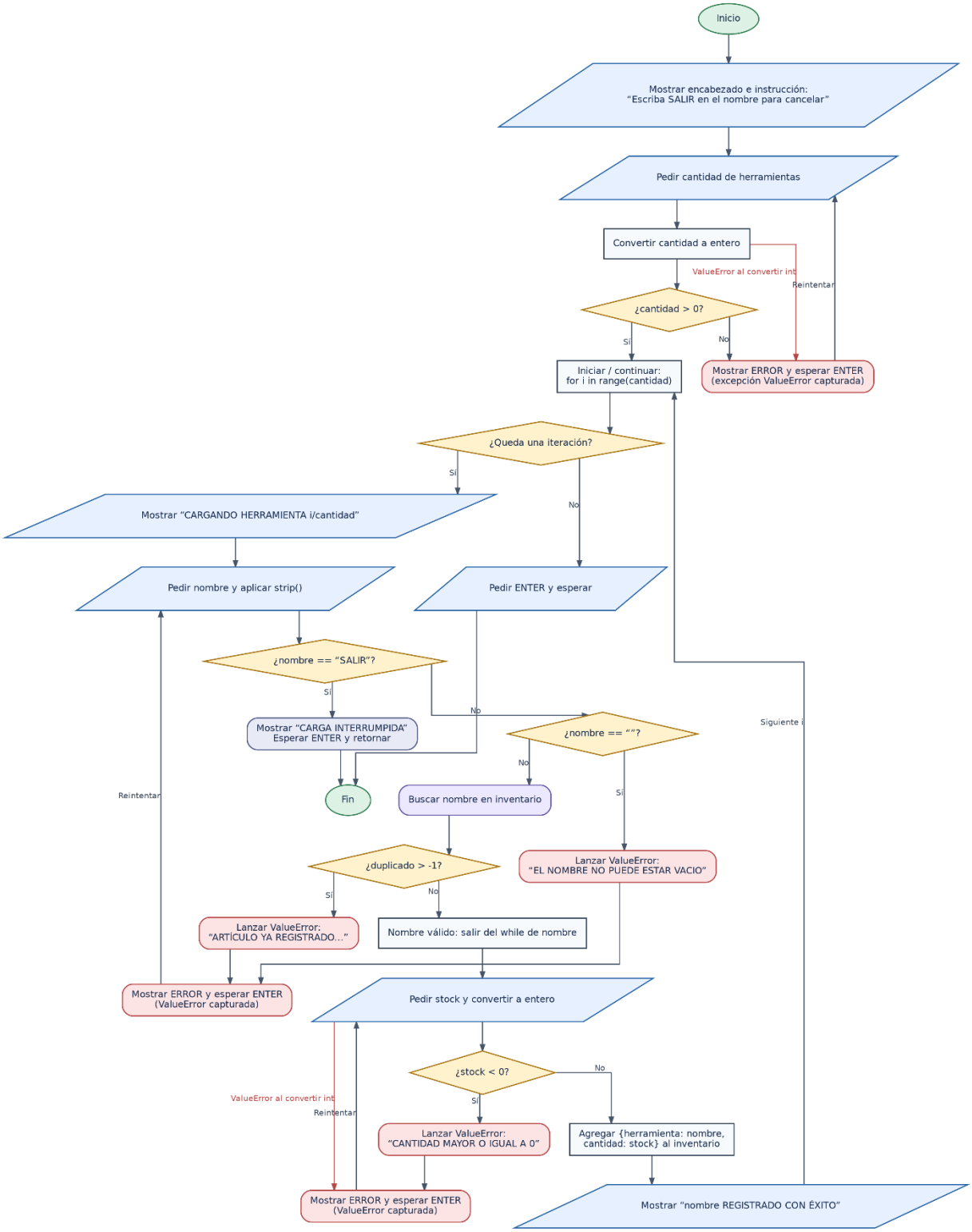
2. **Consulta y Venta:** Llega un cliente buscando un tornillo específico. El usuario ingresa al menú de búsqueda, escribe el nombre y el sistema le muestra en tiempo real cuántas unidades quedan.
3. **Actualización de Stock:** Si se realiza la venta o llega un nuevo pedido del proveedor, el usuario selecciona la opción correspondiente para modificar la cantidad, asegurándose de que el sistema valide que no se introduzcan números negativos.

4. **Cierre:** Al finalizar, el usuario sale del menú y el programa finaliza su ejecución, liberando la memoria utilizada durante la sesión.

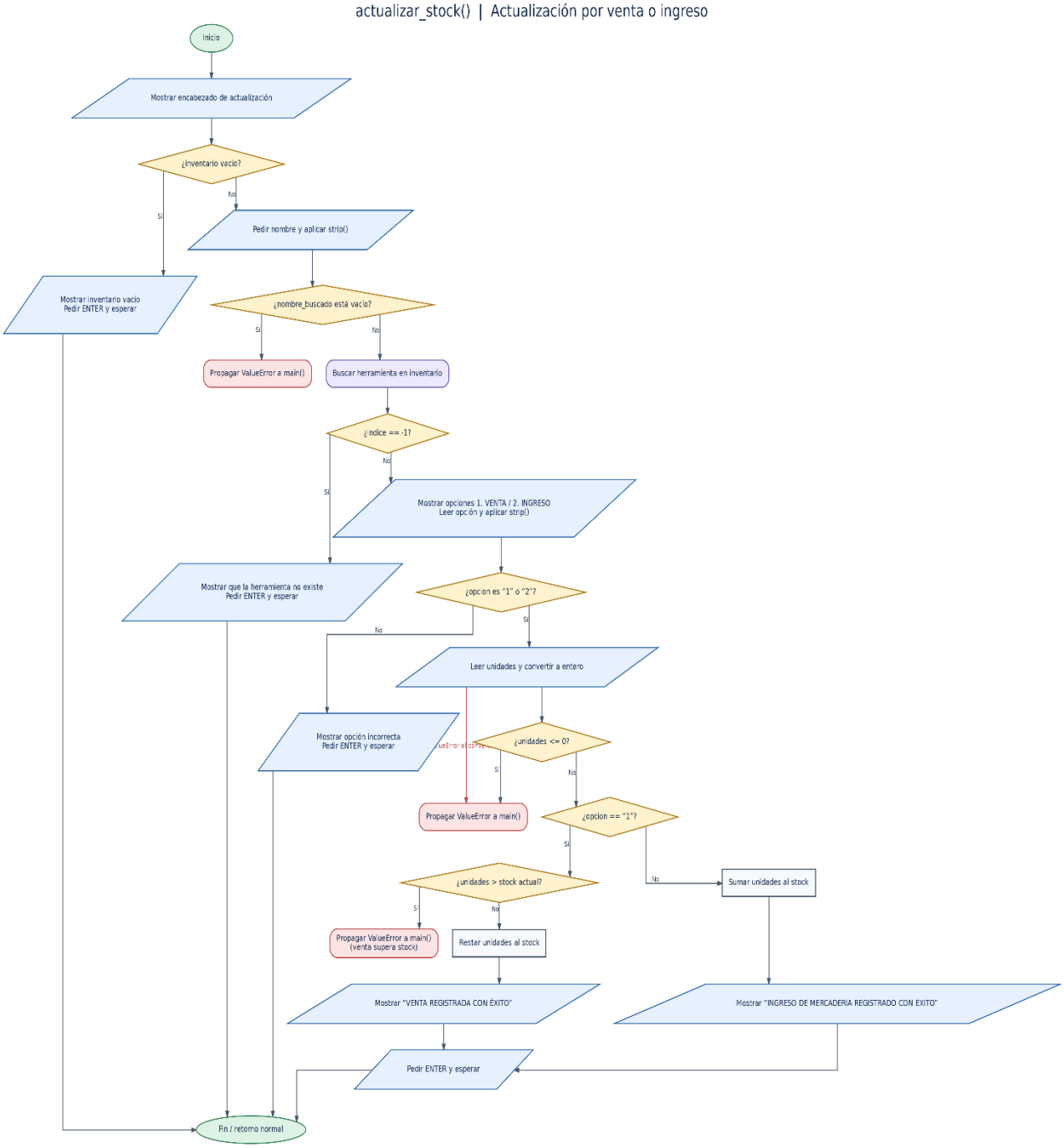
A continuación se mostrará el flujo del programa de manera general, y algunas de sus funciones claves:



cargar_herramientas() | Carga iterativa con validación y reintentos



actualizar_stock() | Actualización por venta o ingreso



Objetivos de la solución de Software:

- Tener un registro actualizado y en tiempo real de las herramientas disponibles
- Generar reportes estadísticos básicos y listados sobre el estado actual de las mercancías
- Prevenir la introducción de datos inconsistentes, nulos, o de tipo incorrectos mediante un sistema estricto de validación en la interfaz
- Garantizar la modularidad del programa haciendo uso de funciones dedicadas (modularizar por medio de funciones dedicadas)

Lo que el software no busca cumplir:

- Este sistema no actuará como un registro de ventas o caja registradora, por lo que excluye el control de ingresos monetarios, facturación o historial de transacciones comerciales.
- No serán utilizadas estructuras de datos avanzadas (sets, deques, etc…)
- No serán utilizadas clases ni objetos (ni nada de POO en general)
- No serán utilizadas variables globales

Marco Teórico y Conceptos Aplicados:

Lista: Según la documentación oficial de Python, una lista se define como una secuencia mutable, utilizada típicamente para almacenar colecciones de elementos. Esto complementa lo establecido por los apuntes de la cátedra, que la definen como una estructura de datos mutable que permite almacenar una secuencia ordenada de elementos. En esta solución, se utilizará una lista principal del programa, que actuará como el contenedor **principal** del almacén. Dentro de ella se alojará de forma secuencial otra estructura (diccionarios), lo que nos permitirá agrupar, recorrer y procesar dinámicamente todo el catálogo de herramientas de la ferretería.

Diccionarios: Según los apuntes de la cátedra, un diccionario almacena pares clave–valor. En lugar de acceder por posición, accedemos por una clave única. Según la documentación oficial de Python, un diccionario es un objeto de mapeo (mapping), que asocia valores hash, a objetos arbitrarios utilizando key – values. Este objeto de datos será utilizado en la solución de la ferretería de una manera específica: un único diccionario que va a albergar dos pares de clave-valor fijas, la clave ‘herramienta’ con su valor siendo de tipo String, y la clave ‘cantidad’, con su valor asociado de tipo int. Cada herramienta nueva se construirá de forma literal y directa, como argumento dentro del método .append()

Funciones: Los apuntes de la cátedra nos indican que las funciones son un bloque de código con nombre que realiza una tarea específica, que puede ser invocado cada vez que se necesite. Complementando esto con la documentación oficial de Python, que especifica que una función es una sentencia ejecutable cuyo efecto es asociar el nombre de la función en el

espacio de nombres local (local namespace) a un objeto de función, el cual encapsula el código ejecutable. Esta definición no ejecuta el cuerpo de la función, solo se procesa cuando la función es explícitamente llamada.

En el programa a realizar para la ferretería, esta definición es el pilar de la modularización obligatoria, cada opción del menú se resolverá mediante una función propia. Teniendo en cuenta que, una función recibe sus parámetros por asignación, al pasarle la lista inventario[] como argumento a la funciones, al ser la lista un objeto mutable, cualquier alteración interna alterará directamente los datos del bloque principal de forma segura, eliminando la necesidad de usar variables globales.

Condicionales: Los apuntes de la cátedra definen a las estructuras condicionales como una porción de código que se ejecuta solo si se cumple cierta condición, lo cual complementa la documentación oficial de Python, que nos indica que la sentencia if es utilizada para la ejecución de condicionales, que selecciona un bloque de código mediante la evaluación de expresiones booleanas una por una hasta que encuentra una que es True y la ejecuta. Si todas las expresiones booleanas resultan ser falsas, el bloque de código la sentencia else se ejecuta en caso de estar presente. Se hará uso de estas estructuras condicionales en el menú principal para bifurcar el flujo hacia la función seleccionada por el usuario y dentro de cada función lógica, para validar que se cumplan las reglas del negocio (como evitar stock negativo o nombres duplicados).

Bucles (Estructuras de Repetición): Son estructuras de control de flujo que permiten ejecutar un bloque de instrucciones múltiples veces de manera automatizada. Esta repetición se mantiene activa mientras se cumpla una condición lógica preestablecida o se recorra una colección de elementos. En el lenguaje Python, las dos estructuras fundamentales son el bucle while (basado en una condición de verdad) y el bucle for (basado en la iteración sobre secuencias).

En el proyecto se utilizaron ambos:

- Bucle while: Se utilizó en el bloque principal (main) para construir un menú interactivo persistente que se repite indefinidamente hasta que el usuario ingresa la opción 7 para salir. También se empleó dentro de cargar_herramientas para encapsular la petición de datos y forzar reintentos locales ante entradas inválidas sin perder el progreso de la carga.
- Bucle for**:** Se aplicó para recorrer secuencialmente la lista de diccionarios inventario. Se utiliza en funciones como buscar_herramienta_inventario para inspeccionar artículo por artículo buscando coincidencias de nombre, y en mostrar_inventario o consultar_agotados para extraer y presentar los datos almacenados de forma ordenada.

Manejo de errores (Excepciones): Las excepciones son mecanismos que interrumpen el flujo normal de un programa para gestionar errores en tiempo de ejecución o condiciones excepcionales. Una excepción puede ser provocada nativamente por el intérprete (ante un error de ejecución) o ser lanzada de forma explícita por el programador mediante la sentencia raise. Para capturarlas y procesarlas, se utiliza la estructura try ... except. Python emplea un modelo

de "terminación": el bloque que captura el error puede continuar la ejecución en un nivel exterior, pero no puede reparar la causa original ni reintentar la operación fallida de forma automática, a menos que se vuelva a ingresar al bloque de código desde el inicio mediante un bucle. Si una excepción no es capturada, el programa detiene su ejecución por completo.

En este sistema, el manejo de errores se implementó siguiendo dos estrategias arquitectónicas diferenciadas según los requerimientos de robustez:

- **Captura Local para Reintento (Opción 1):** En cargar_herramientas, se envuelven las entradas numéricas y de texto en bloques try/except dentro de bucles while True. Si el usuario ingresa un dato inválido, la excepción se captura localmente para informar el error y forzar el reintento inmediato sin perder el progreso de la carga general.
- **Excepciones Viajeras (Opciones 3, 5 y 6):** En funciones como consultar_stock, registrar_producto y actualizar_stock, se validan las reglas de negocio (nombres vacíos, artículos duplicados, stocks negativos o ventas excedidas) y se lanza manualmente un raise ValueError con un mensaje descriptivo. Al no poner un except dentro de estas funciones, la excepción interrumpe la ejecución de la función, "viajando" hacia el bloque principal main(). El main() captura el error, muestra el motivo en pantalla y devuelve al usuario de forma segura al menú principal, cumpliendo estrictamente con la arquitectura modular exigida.

Propuesta de arquitectura técnica:

El sistema se estructura en un único archivo fuente de Python denominado **Reta_Victor_P1.py**, el cual debe ser ejecutado a través de la función main() para iniciar el sistema. Se diseñó bajo un enfoque modular y procedimental, eliminando por completo el uso de variables globales y clases según las restricciones del proyecto. El flujo del programa se divide en tres componentes clave:

- **Capa de Control Central (Bloque Principal):** El main() actúa como el orquestador del programa. Contiene el menú interactivo persistente, evalúa la opción elegida y centraliza la captura de excepciones (try/except) de las **opciones 3, 5 y 6** para gestionar el retorno seguro al menú ante datos inválidos.
- **Capa de Lógica de Negocio (Funciones Dedicadas):** Un conjunto de funciones específicas (def) que reciben el estado del inventario por parámetro. Aplican las reglas de negocio, validan las entradas mediante condicionales (if/elif/else) y disparan excepciones (raise ValueError) cuando se violan las restricciones de stock o nombres.
- **Capa de Datos**: Representada exclusivamente por la lista de diccionarios inventario. Los datos mutan "en tiempo real" dentro de las funciones asignadas debido al paso de parámetros por referencia, persistiendo solo mientras el programa se encuentre en ejecución.

Para respetar la modularidad exigida por la cátedra, se estructuró el programa en las siguientes funciones:

main() -> None: Actúa como el bloque principal y orquestador del programa. Inicializa la lista vacía inventario y ejecuta un bucle while True persistente donde invoca a imprimir_menu() y pide la opción mediante input(), usando un código de escape (\033[7D) para posicionar el cursor entre los corchetes. Todo el flujo se procesa dentro de un bloque try/except. Intenta convertir la opción a entero con int() y la evalúa mediante un condicional if/elif/else: redirige a las funciones de las opciones 1 a 6 pasando inventario como parámetro; interrumpe el ciclo con break si es la opción 7; o lanza un raise ValueError si el número está fuera de rango. El bloque except ValueError centraliza la gestión de errores, capturando tanto los fallos nativos de conversión de texto a entero como las excepciones viajeras propagadas desde las opciones 3, 5 y 6, imprimiendo el mensaje correspondiente y garantizando el retorno seguro al menú interactivo. No retorna ningún valor.

buscar_herramienta_inventario(inventario: list, nombre: str) -> int: Función auxiliar de búsqueda. Recibe como parámetros la lista de diccionarios creada en la función main(), y un nombre buscado. Internamente normaliza el nombre buscado y el nombre de la herramienta en la lista con .upper() para eliminar cualquier uso de minúsculas o mayúsculas introducidas por el usuario, y el uso de .strip() para eliminar espacios, y, si coinciden, se devuelve un número entero con la posición donde fue encontrado en la lista, o un **-1** en caso de no haber encontrado nada.

def imprimir_menu()-> None: Función que se encarga de mostrar de forma visual en la consola las opciones disponibles del sistema interactivo. No recibe ningún parámetro y hace uso exclusivo de la función nativa print() para renderizar las siete opciones del menú ordenadas numéricamente (de la opción 1 a la opción 7). Sirve como interfaz gráfica básica para guiar al usuario en la selección de operaciones del programa y no retorna ningún tipo de dato (None).

def cargar_herramientas(inventario: list) -> None: Permite la carga de herramientas de forma masiva e iterativa. Dentro de un bucle while, solicita al usuario la cantidad de herramientas a registrar; si se ingresa un valor menor o igual a cero, lanza un ValueError que es capturado por un bloque except local, el cual informa el error en la consola y reinicia el bucle para solicitar nuevamente el dato. Una vez superado este bloque, el programa ingresa a un bucle for que se repite de acuerdo a la cantidad indicada. Para cada elemento, solicita el nombre de la herramienta. Se implementó la opción de interrumpir la carga si el usuario escribe "SALIR", retornando de manera temprana a la función principal main(). Posteriormente, verifica si el nombre está vacío; en caso afirmativo, lanza un ValueError capturado localmente para forzar el reintento. Luego, invoca a la función auxiliar buscar_herramienta_inventario(inventario, nombre); si esta retorna un valor mayor a -1, lanza un ValueError por artículo duplicado. Finalmente, mediante otro bucle de reintento local, solicita el stock inicial y valida con try/except y raise que sea un entero mayor o igual a cero; cumplidas todas las condiciones, añade el diccionario correspondiente a la lista principal mediante el método .append().

def mostrar_inventario(inventario: list)-> None: Muestra en la consola los datos guardados en la lista de diccionarios si la longitud de la lista es mayor a 0, haciendo uso de la función len(). En caso de que la longitud de la lista sea 0, imprime un mensaje por pantalla indicando que la lista está vacía y retorna a la función principal main. Si la longitud es mayor a 0, recorre cada elemento de la lista, e imprime los pares key: value que están almacenados.

def consultar_stock(inventario: list)-> None: Si la longitud de la lista recibida es 0, la función imprime en consola que el inventario está vacío y retorna al menú principal. Si la longitud de la lista es mayor a 0, se le pide al usuario que ingrese el nombre de la herramienta que busca. Si la longitud del nombre introducido por el usuario es 0, entonces el programa lanza un valueError con raise, que viaja hasta la función principal main, donde se captura e imprime en consola. En caso de que la longitud del nombre ingresado por el usuario sea mayor que 0, llama a la función **buscar_herramienta_inventario(inventario, nombre)** y le pasa el inventario y el nombre. En caso de que esta función retone un valor > -1, la función lo imprime en pantalla junto con su stock, caso que la función retorne -1, la función imprime en consola que el artículo no existe en el inventario, y retorna a la función principal main.

def consultar_agotados(inventario: list)-> None: Evalúa inicialmente la longitud de la lista. Si es 0, el programa imprime un mensaje en la consola y retorna a la función principal main(). Si la longitud de la lista de diccionarios es mayor a 0, recorre toda la estructura de datos con un bucle for y un condicional consultando el valor 'cantidad' en cada diccionario. Si la cantidad es 0, se imprime por consola el nombre de la herramienta y se incrementa el contador_agotados en 1 unidad. Al finalizar el bucle for, si el contador_agotados es igual a 0, imprime esto por consola y retorna a la función main. En caso que contador_agotados sea mayor a 0, imprime por consola el valor de contador_agotados y retorna a la función main.

def registrar_producto(inventario: list)-> None: Le pide al usuario que ingrese el nombre de la herramienta que quiere registrar. Si la longitud del nombre de la herramienta es 0, lanza un ValueError mediante la sentencia raise, lo que detiene la ejecución de la función y delega la captura y la impresión del error a la función main. Si el nombre es válido, invoca a la función auxiliar **buscar_herramienta_inventario(inventario, nombre).** Si la función auxiliar retorna un valor mayor a -1, el programa lanza un ValueError con raise que viaja hasta main. Se le pide también al usuario que ingrese el stock del producto. Si el stock ingresado por el usuario es menor a 0, se lanza una nueva excepción ValueError hacia el bloque principal main. Finalmente, si se superan todas las validaciones, se agrega a la lista como último elemento con el método .append(), utilizando el nombre y el stock introducidos por el usuario como argumentos.

def actualizar_stock(inventario: list) -> None: Si la longitud de la lista que recibe la función es 0, el programa imprime por pantalla un mensaje y retorna a la función main. Se le pide al usuario que ingrese el nombre de la herramienta buscada. Si la longitud del nombre de la herramienta es 0, se lanza un ValueError con raise que delega la captura del error y mostrar el mensaje a la función main. Luego, se hace uso de la función auxiliar.

buscar_herramienta_inventario(inventario, nombre). Si la función auxiliar retorna -1, se le indica al usuario que la herramienta no existe en el inventario y se retorna a la función principal main. Posteriormente, se le pide al usuario que ingrese una opción para indicar si el movimiento se trata de una venta (1) o de un ingreso de inventario (2). Si la entrada es incorrecta, se emite un mensaje y retorna al bloque principal main(). Luego, solicita las unidades del movimiento, lanzando un ValueError con raise en caso de que el numero ingresado sea menor o igual a 0. Si el movimiento corresponde a una venta, verifica mediante un condicional que las unidades que se quieren vender no superen el stock disponible de la herramienta, de lo contrario, dispara un raise ValueError hacia el main(). Cumpliendo todas

estas validaciones, actualiza el stock (restando en caso de venta, o sumando en caso de ingreso) y se muestra su respectivo mensaje en la consola.

Entorno y Recursos de Desarrollo:

- **Lenguaje de Programación:** Se usó Python 3.14.5 ya que es el estándar requerido por la cátedra, ideal por su sintaxis y por el enfoque modular que le dio al proyecto.
- **Entorno de desarrollo (IDE):** Se usó Visual Studio Code como editor de texto ya que su terminal integrada facilita las pruebas manuales y brinda extensiones para depuración de código de Python.
- **Recursos de desarrollo:** la suite ofimática Word para la redacción del presente informe escrito.
- **Archivo de Inicio:** El sistema está estructurado en un único módulo fuente, siendo **Reta_Victor_P1.py** el archivo que debe ejecutarse para iniciar el programa, a través de la función main().

Fuentes:

How to write a good software design doc: https://www.freecodecamp.org/news/how-to-write-a-good-software-design-document-66fcf019569c/

Listas: https://docs.python.org/3/builtins/stdtypes.html#typesseq-list

Diccionarios: https://docs.python.org/3/builtins/stdtypes.html#dict

Funciones: https://docs.python.org/3/reference/compound_stmts.html#function

Condicionales: https://docs.python.org/3/reference/compound_stmts.html#the-if-statement

Bucles: Libro Think Python How to Think Like a Computer Scientist, by: Allen Downey, capítulo 7: Iteration

Manejo de errores (Excepciones): https://docs.python.org/3/reference/executionmodel.html#exceptions